4.2 Analysis of Robotic Arm Underlying Program

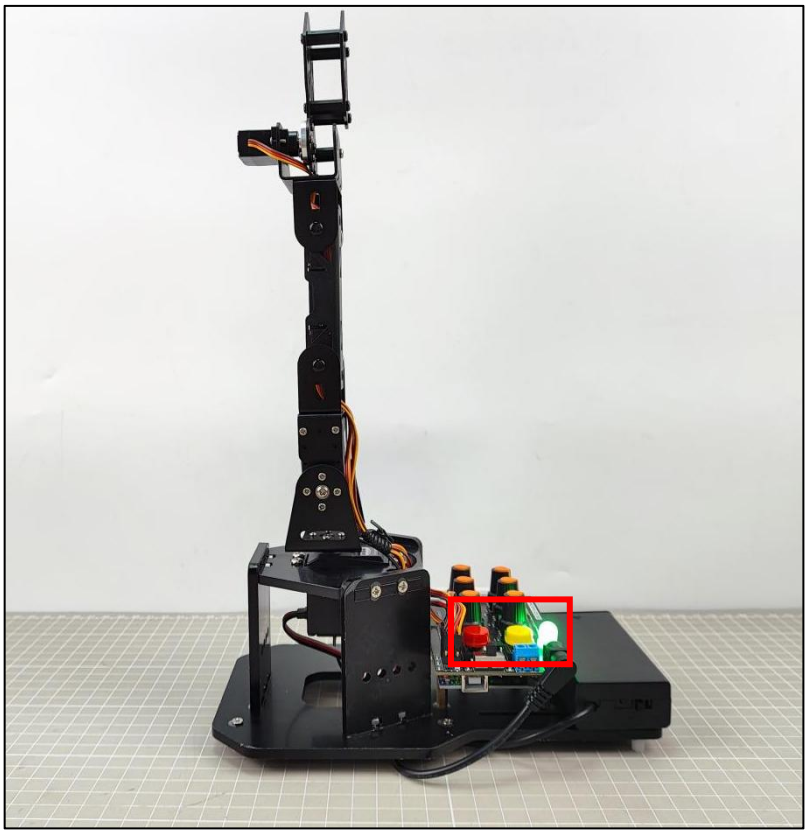
This section demonstrates how to implement the control program function of robotic arm. You can deepen the understanding of MiniArm underlying program through the analysis of servo neutral function, Bluetooth handling function, knob reading function, servo control function, buzzer-related function, action group execution function, and button handling function.

MiniArm comes with the underlying program pre-burned by default. Please check the Appendix: MiniArm Underlying Program Burning if you need to re-burn it.

1. Program Running Outcome

1)After powering on, the robotic arm returns to the neutral position while the RGB on the expansion board lights up white, which is the neutral program.

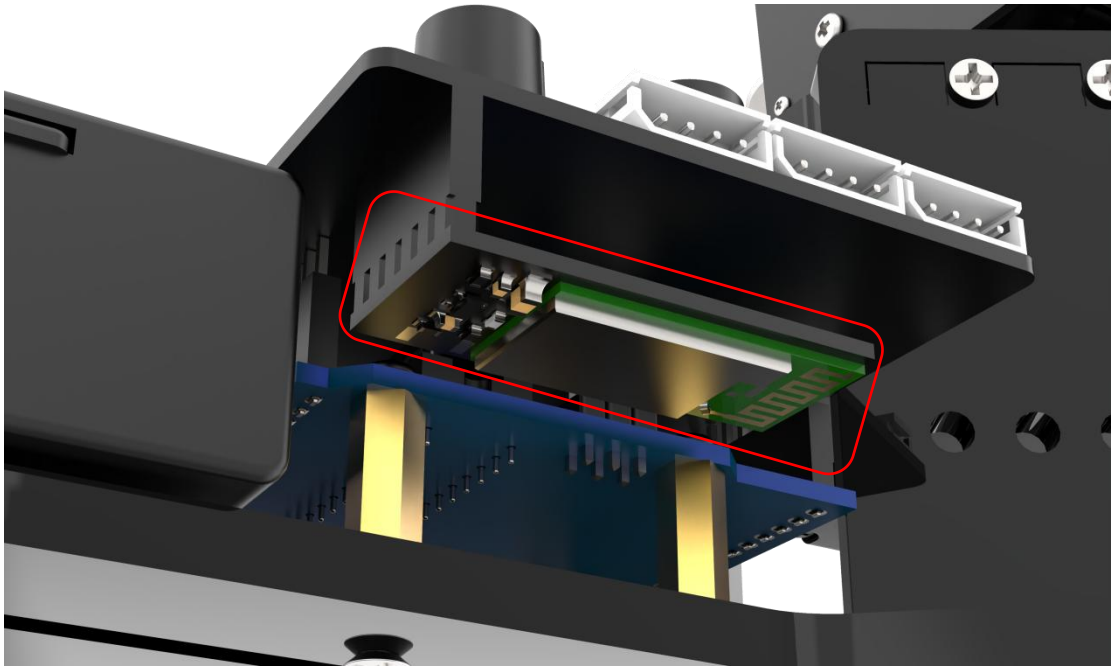
2)Press two buttons at the same time for one second. Enter the knob control program when the RGB lights up green, which means unlock the neutral function.



3)Long press button K1 to enter the action group editing mode. This motion will clear previously edited action group at first, then control the position of robotic arm by knobs. Press K1 to record this position into the action group. Finally, long press K1 to exit action group editing mode and preserve edited action group to the Flash. Then short press K2 to run the action group.

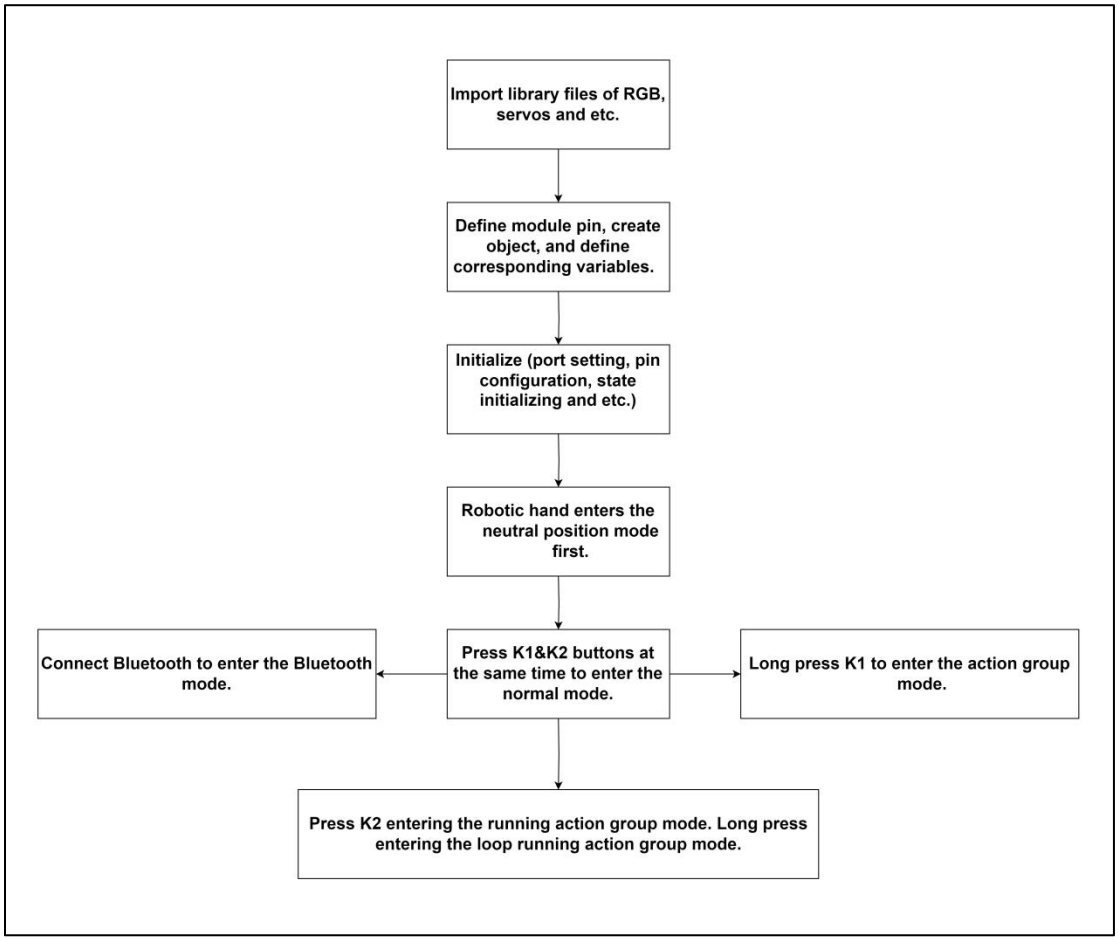
You can check “2. Offline Knob Programming” for detailed button instruction.

4)When the Bluetooth module is plugged in and connected, the robotic arm will enter Bluetooth control program.



2. Program Brief Analysis

The name of this program is “MiniArm.ino”. For the implementation logic of the program, you can refer to the following flowchart:



2.1 Servo Neutral Function “servos_middle()”

“servos_middle()” is a function that puts all servos in the middle position, which is executed within the “set_up()” function. To produce a neutral installation environment, it can be removed in the subsequent program.

The procedure is as follows:

①Read the current deviation angle;

②Set all the servos to the neutral angle;

③Detect current button state in a loop:

1)Press K1 button for more than two seconds to enter deviation adjustment. At this point, you can rotate the knobs to return the servo to the initial position. Then long press K2 for more than two seconds,

the adjusted position will be preserved.

2)When two buttons are pressed at the same time for about two seconds, the loop will exit and the function will exit;

3)When two buttons are not pressed at the same time or the count of pressed buttons does not reach two seconds, the count is cleared and the loop will continue.

```
void servos_middle(void)
{
  // read deviation angle
  for (int i = 0; i < 16; ++i) {
    eeprom_read_buf[i] = EEPROM.read(EEPROM_SERVO_OFFSET_START_ADDR + i);
  }
  if (strcmp(eeprom_read_buf, EEPROM_START_FLAG) == 0) {
    // Serial.println("EEPROM read");
    memset(eeprom_read_buf,  0 , sizeof(eeprom_read_buf));
    for (int i = 0; i < 6; ++i) {
      eeprom_read_buf[i] = EEPROM.read(EEPROM_SERVO_OFFSET_DATA_ADDR + i);
    }
    memcpy(servo_offset , eeprom_read_buf , 6);
  }

  int16_t set_angles = 0;
  // set the servo to the neutral angle
  for (int i = 0; i < 5; ++i) {
    set_angles = 90;
    set_angles = set_angles < limt_angles[i][0] ? limt_angles[i][0] : set_angles;
    set_angles = set_angles > limt_angles[i][1] ? limt_angles[i][1] : set_angles;
    servos[i].write(i == 0 || i == 5 ? 180 - set_angles : set_angles);
  }
```

```
uint16_t step = 0 , i = 0 , count[2] = {0,0};
// test in a loop, long press two buttons at the same time to exit the neutral position task
while(true)
{
  for(i = 0 ; i < 2 ; i++)
  {
    if(!digitalRead(keyPins[i]))
      count[i] = count[i] > 500 ? 500 : count[i]+1;
    else
      count[i] = 0;
  }
  switch(step)
  {
    case 0: //get button value
      if(count[0] > LONG_PRESS && count[1] > LONG_PRESS-50)
      {
        // calculate deviation
        for(i = 0; i < 6 ; i++){
          servo_offset[i] = knob_angles[i] - 90 + servo_offset[i];
        }
        for (int j = 0; j < strlen(EEPROM_START_FLAG) + 1; ++j) { /* storage space initialized flag */
          EEPROM.write(EEPROM_SERVO_OFFSET_START_ADDR + j, EEPROM_START_FLAG[j]);
        }
        memcpy(eeprom_read_buf , (uint8_t*)servo_offset , 6);
        // memset(eeprom_read_buf , 0 , sizeof(eeprom_read_buf)); //clear offset
        for (int k = 0; k < 6; ++k) {// preserve deviation
          EEPROM.write(EEPROM_SERVO_OFFSET_DATA_ADDR + k, eeprom_read_buf[k]);
        }
        return;
```

```
      }else if(count[0] > LONG_PRESS) //Long press K1 alone to jump into offset adjustment
      {
        step = 1;
        FastLED.clear(); //green
        rgbs[0].g = 100;
        FastLED.show();
      }
      break;

    case 1: //adjust deviation
      knob_update(); // knob update
      servo_control(); // servo control
      if(count[1] > LONG_PRESS) //finish adjustment, preserve deviation
      {
        FastLED.clear(); //white
        rgbs[0].r = 100;
        rgbs[0].g = 100;
        rgbs[0].b = 100;
        FastLED.show();
        step = 0;
      }
      break;

    default:
      step = 0;
      break;
  }
  delay(10);
}
}
```

2.2 Bluetooth Handling Function “recv_handler()”

“recv_handler()” is a handling function that receive Bluetooth serial port

information resolution and control the robotic arm.

Program analysis:

Read serial port information until encountering the '$' symbol, then stop reading and check the first character:

① When the first character is 'A' ~ 'F', the corresponding arm servo movement is controlled;

```
void recv_handler(void) {
  while (Serial.available() > 0) {
    String cmd = Serial.readStringUntil('$');
    switch (cmd[0]) {
      case 'A':
        app_angles[0] = atoi(cmd.c_str() + 1);
        g_mode = MODE_APP;
        break;
      case 'B':
        app_angles[1] = atoi(cmd.c_str() + 1);
        g_mode = MODE_APP;
        break;
      case 'C':
        app_angles[2] = atoi(cmd.c_str() + 1);
        g_mode = MODE_APP;
        break;
      case 'D':
        app_angles[3] = atoi(cmd.c_str() + 1);
        g_mode = MODE_APP;
        break;
      case 'E':
        app_angles[4] = atoi(cmd.c_str() + 1);
        g_mode = MODE_APP;
        break;
      case 'F':
        app_angles[5] = atoi(cmd.c_str() + 1);
        g_mode = MODE_APP;
        break;
```

②When the first character is 'G' ~ 'J', the color of RGB light is controlled;

```
case 'G':
  g_mode = MODE_APP;
  rgbs[0].r = atoi(cmd.c_str() + 1);
  break;
case 'H':
  g_mode = MODE_APP;
  rgbs[0].g = atoi(cmd.c_str() + 1);
  break;
case 'I':
  g_mode = MODE_APP;
  rgbs[0].b = atoi(cmd.c_str() + 1);
  break;
case 'J':
  g_mode = MODE_APP;
  FastLED.show();
  break;
```

③When the first character is ‘Z’, the buzzer sound is controlled.

```
    case 'Z':
      {
        g_mode = MODE_APP;
        if (cmd[1] == '1') {
          play_tune(DOC6, 10000u, 1u);
        }
        if (cmd[1] == '0') {
          play_tune(DOC6, 1u, 0u);
        }
        break;
      }
    default:
      break;
  }
}
```

2.3 Knob Reading Function “knob_update()”

“knob_update()” is a function that reads values of 5 knob potentiometers, and controls the position of the corresponding servo based on this value.

The program logic is as follows:

① Read the analog value of each potentiometer, and obtain the expected value of the knob by weighted calculation.

②Map the expected value of the knob to the range of 0 to 180 degrees of the servo, read the current knob value and store it.

③If the difference between the current knob value and the next read value is 5, it will enter the knob mode. For example, in the current app mode, when you rotate the knob a difference will occur. At this time, the robotic arm will enter the knob mode and will not disconnect with the app. Using the app again will return to app control mode. If it is not rotated and the knob value does not change, it will not enter the knob mode.

④If currently in knob mode, assign the servo angle mapped to the current value of the knob to the servo. Then the servo will rotate to the corresponding angle.

```
void knob_update(void) { /* Knob reading update */
  static uint32_t last_tick = 0;
  static float values[6];
  float angle = 0;
  if (millis() - last_tick < 10) {
    return;
  }
  last_tick = millis();
  values[0] = values[0] * 0.7 + analogRead(A0) * 0.3;
  values[1] = values[1] * 0.7 + analogRead(A1) * 0.3;
  values[2] = values[2] * 0.7 + analogRead(A2) * 0.3;
  values[3] = values[3] * 0.7 + analogRead(A3) * 0.3;
  values[4] = values[4] * 0.7 + analogRead(A4) * 0.3;
  values[5] = values[5] * 0.7 + analogRead(A5) * 0.3;
  for (int i = 0; i < 6; ++i) {
    angle = map(values[i], 0, 1023, 0, 180);
    angle = angle < 0 ? 0 : (angle > 180 ? 180 : angle);
    if (fabs(angle - knob_angles[i]) > 5 && g_mode != MODE_KNOB) {
      /* When it is found that the knob has been rotated beyond the threshold,
      the knob control mode will be restored (possibly in mobile app control mode) */
      g_mode = MODE_KNOB;
      action_group_running_step = 0;
      rgbs[0].r = 0;
      rgbs[0].g = 255;
      rgbs[0].b = 0;
      FastLED.show();
    }
    if (g_mode == MODE_KNOB) {
      knob_angles[i] = angle;
    }
  }
}
```

2.4 Servo Control Funtion “servo_control()”

“servo_control()” is a function that actually controls servo. This function enters every 25ms. Depending on the mode, it calculates the actual angle value for each servo and ultimately assigns it to the PWM control function to control the servo angle.

Because servo No.0 has the opposite mechanical angle compared to the other servos, a flip needs to be performed here.

```
void servo_control(void) {
  static uint32_t last_tick = 0;
  static uint8_t tmp_angles = 0;
  if (millis() - last_tick < 20) {
    return;
  }
  last_tick = millis();
  for (int i = 0; i < 5; ++i) {
    if (g_mode == MODE_APP) {
      tmp_angles = app_angles[i];
    } else if (g_mode == MODE_KNOB) {
      tmp_angles = knob_angles[i];
    } else if (g_mode == MODE_EXTENDED) {
      tmp_angles = extended_func_angles[i];
    } else{
      tmp_angles = action_angles[i];
    }

    if(servo_expect[i] > tmp_angles)
    {
      servo_expect[i] = servo_expect[i] * 0.9 + tmp_angles * 0.1;
      if(servo_expect[i] < tmp_angles)
        servo_expect[i] = tmp_angles;
    }else if(servo_expect[i] < tmp_angles)
    {
      servo_expect[i] = servo_expect[i] * 0.9 + (tmp_angles * 0.1 + 1);
      if(servo_expect[i] > tmp_angles)
        servo_expect[i] = tmp_angles;
    }

    servo_angles[i] = servo_expect[i] + servo_offset[i];

    servo_angles[i] = servo_angles[i] < limt_angles[i][0] ? limt_angles[i][0] : servo_angles[i];
    servo_angles[i] = servo_angles[i] > limt_angles[i][1] ? limt_angles[i][1] : servo_angles[i];
    servos[i].write(i == 0 || i == 5 ? 180 - servo_angles[i] : servo_angles[i]);
  }
}
```

2.5 Buzzer-Related Function

(1) “tune_task()” is buzzer sound task function. According to the parameters set by the “play_tune()” function, the programmed tone arrays is played sequentially.

```
void tune_task(void) {
  static uint32_t l_tune_beat = 0;
  static uint32_t last_tick = 0;
  // if it is not timed  and  it rings the same number as last time
  if (millis() - last_tick < l_tune_beat && tune_beat == l_tune_beat) {
    return;
  }
  l_tune_beat = tune_beat;
  last_tick = millis();
  if (tune_num > 0) {
    tune_num -= 1;
    tone(buzzerPin, *tune++);
  } else {
    noTone(buzzerPin);
    tune_beat = 10;
    l_tune_beat = 10;
  }
}
```

//if it is not timed and it rings the same number as last time

(2)“play_tune()” function is an interface function used for setting the buzzer sound. You can call this function to set buzzer sound. Parameter 1 means the array of tones to be played; parameter 2 means the duration for each tone sounding; parameter 3 means the number of tones contained within the array.

```
void play_tune(uint16_t *p, uint32_t beat, uint16_t len) {
  tune = p;
  tune_beat = beat;
  tune_num = len;
}
```

2.6 Action Group Running Function “action_group_task()”

“action_group_task()” function is used for executing action groups, which is running in the “loop()” circulation.

The function logic is as follows:

①When the “action_group_running_step” is 0, the action group will not be executed.

② When the “action_group_running_step” function is 1 or 2, the action group will start running the action group. First, all actions within the action group are read. Then, determine whether there is an action in the action group. If there is an action, assign “action_group_running_step” to 3~5, and the program proceeds to the next step for execution.

```
switch (action_group_running_step) {
  case 0:
    break;
  case 1:
  case 2:
    {
      g_mode = MODE_ACTIONGROUP;
      action_index = 0;
      action_num = 0;
      for (int i = 0; i < 16; ++i) {
        eeprom_read_buf[i] = EEPROM.read(i);
      }
      if (strcmp(eeprom_read_buf, EEPROM_START_FLAG) == 0) {
        action_num = EEPROM.read(EEPROM_ACTION_NUM_ADDR);
        if (action_num > 0) {
          tick_wait = 1000;
          action_index = 0;
          if (action_group_running_step == 1) {
            action_group_running_step = 3;
          }
          if (action_group_running_step == 2) {
            action_group_running_step = 4;
          }
        } else {
          action_group_running_step = 0;
        }
      } else {
        action_group_running_step = 0;
      }
      break;
    }
```

③Run each read action sequentially until the action group execution is complete. Then assign “action_group_running_step” to 6 and the program proceed to the next step.

```
case 3:
case 4:
case 5:
  {
    memset(eeprom_read_buf, 0, 16);
    for (int i = 0; i < EEPROM_ACTION_UNIT_LENGTH; ++i) {
      eeprom_read_buf[i] = EEPROM.read(EEPROM_ACTION_START_ADDR +
       EEPROM_ACTION_UNIT_LENGTH * action_index + i);
    }
    for (int i = 0; i < 5; i++) {
      action_angles[i] = eeprom_read_buf[i];
    }
    action_index += 1;
    if (action_index >= action_num) {
      if (action_group_running_step == 4) {
        action_group_running_step = 4;
        action_index = 0;
      } else {
        action_group_running_step = 6;
      }
    }
    break;
  }
```

④ Illuminate the RGB light with green and jump back to the first step, stopping the action group.

```
case 6:
  action_group_running_step = 0;
  tick_wait = 50;
  rgbs[0].r = 0;
  rgbs[0].g = 255;
  rgbs[0].b = 0;
  FastLED.show();
  break;
default:
  break;
```

2.7 Button Handling Function “key_scan()”

“key_scan()” function is used for scanning button signal and calling corresponding program. The program flow is as follows:

① Read the button state firstly. Then enter state machine to call the corresponding state.

```
uint16_t io = digitalRead(keyPins[i]);
```

②When in the released state, no action is taken.

```
case 0:
  { /* Release in normal state */
    if (state) {
      key_step[i] = 1;
      pressed_tick[i] = last_tick;
    }
    break;
  }
```

③When short press K1, it is checked whether the system is in action group editing mode. If it is in the editing state, the arm will sounds once and you can add new motion. If it is not in the editing state, the arm stops running the action group and sounds a long beep.

```
if (i == 0)  //K1
{
  if (learning) { /* add new motion */
    memcpy(&action_group[action_index++], knob_angles, 6);
    play_tune(DOC5, 100, 1);
  } else { /* stop running action group */
    if (action_group_running_step != 0) {
      action_group_running_step = 0;
      play_tune(DOC6, 800, 1);
      rgbs[0].r = 0;
      rgbs[0].g = 255;
      rgbs[0].b = 0;
      FastLED.show();
    }
  }
}
```

When short press k2 and it is not in the action group editing mode, run action group and sound a beep.

```
if (i == 1)  //K2
{
  if (!learning) { /* run the action group at once */
    if (action_group_running_step == 0) {
      action_group_running_step = 1;
      play_tune(DOC6, 100u, 1u);
      rgbs[0].r = 255;
      rgbs[0].g = 200;
      rgbs[0].b = 0;
      FastLED.show();
    }
  }
}
```

④ When long press K1, preserve the action group if it is in the action editing mode. Open action group editing mode if it is not in this mode.

```
} else {
  if (last_tick - pressed_tick[i] > 1000)  //If long press
  {
    key_step[i] = 2;
    if (i == 0) {
      if (learning) {                          /* exit action editing mode without savinf   */
        for (int j = 0; j < action_index; ++j) { /* preserve the action group to EEPROM */
          for (int k = 0; k < 6; ++k) {
            EEPROM.write(EEPROM_ACTION_START_ADDR + EEPROM_ACTION_UNIT_LENGTH * j + k, action_group[j][k]);
          }
        }
        /* preserve the number of actions contained in the action group */
        EEPROM.write(EEPROM_ACTION_NUM_ADDR, action_index);

        /* storage space initialized flag */
        for (int j = 0; j < strlen(EEPROM_START_FLAG) + 1; ++j) {
          EEPROM.write(0 + j, EEPROM_START_FLAG[j]);
        }
        learning = false;
        play_tune(MI_RE_DO, 150, 3);
        rgbs[0].r = 0;
        rgbs[0].g = 255;
        rgbs[0].b = 0;
        FastLED.show();
      } else { /* enter action editing mode */
        if (action_group_running_step == 0) {
          learning = true;
          action_index = 0;
          play_tune(DO_RE_MI, 150, 3);
          rgbs[0].r = 255;
          rgbs[0].g = 0;
          rgbs[0].b = 0;
          FastLED.show();
        }
      }
    }
```

When long press K2, exit the action group editing mode and sound a short beep if it is in this mode. If it is not in this mode, run the action group in a loop.

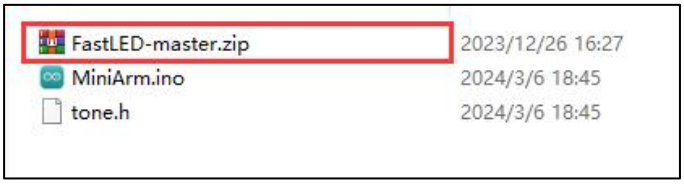
```
if (i == 1) {
  if (learning) { /* exit the action editing mode and preserve it  */
    learning = false;
    play_tune(MI_RE_DO, 150, 3);
    rgbs[0].r = 0;
    rgbs[0].g = 255;
    rgbs[0].b = 0;
    FastLED.show();

  } else {
    if (action_group_running_step == 0) { /* run the action group in a loop */
      play_tune(DOC6, 300u, 1u);
      rgbs[0].r = 255;
      rgbs[0].g = 200;
      rgbs[0].b = 0;
      FastLED.show();
      action_group_running_step = 2; /* run in a loop */
    }
  }
}
```

Appendix: MiniArm Underlying Program Burning

1. Preparation and Notice

Import the .zip library file archive in the same directory as this section to the Arduino IDE according to the “1. Tutorials\4. Secondary Development Basics and Environment Configuration\4.1 Arduino Development Environment Setup\02 Arduino IDE User Instruction” course. You can skip this step if FastLED-master.zip library file has been imported before.



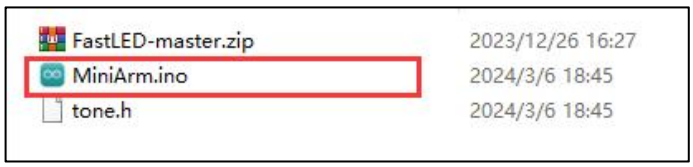
2. Program Download

Note:

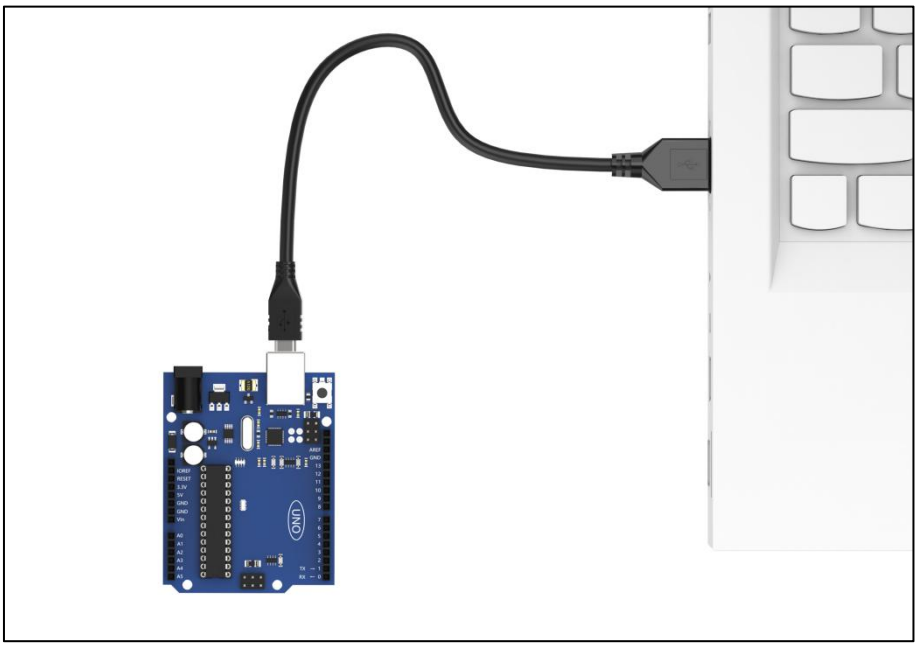
◆ Remove the Bluetooth module before downloading the program, otherwise the serial port conflict will lead to download failure.

◆ Please turn the switch of battery box to “OFF” when connect to Type-B cable to avoid download cable touch the power pin of expansion board by mistake, which may cause a short circuit.

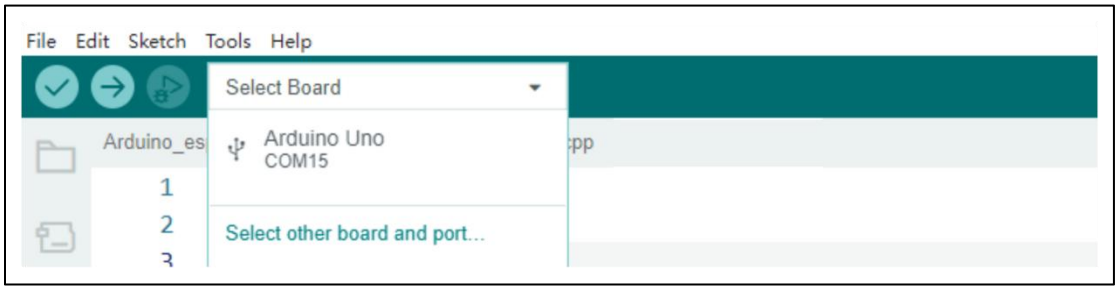
1)Locate and open “MiniArm\MiniArm.ino” program file in the same directory as this section.



2)Connect Arduino to the computer through UNO Type-B cable.



3)Click “Select Development Board” option. The software will automatically test current Arduino serial. Then click to connect it.



4)Click to download the program to Arduino, and wait for finishing download.